

Retos de programación

Reto 1. Batalla de dados

Vamos a crear un pequeño juego en el que te enfrentarás al ordenador en una batalla de dados.

El programa debe lanzar un dado para el jugador y otro para el ordenador.

Python tiene una librería llamada `random` que permite generar valores aleatorios, por ejemplo, para simular una tirada de dados.

- Primero tenemos que importarla: `import random`
- Después podemos utilizar `randint()` para obtener un número entero aleatorio entre dos valores, incluyendo ambos extremos: `numero = random.randint(1, 6)`. En este caso, `numero` puede valer aleatoriamente 1, 2, 3, 4, 5 o 6, como si hubiéramos lanzado un dado. Cada vez que ejecutemos el programa, Python puede obtener un resultado diferente.

Reglas

- Cada dado genera un número entre **1 y 6**.
- El resultado obtenido se convierte en puntos:
 - Si sacas un **6**, obtienes **8 puntos**.
 - Si sacas un **1**, obtienes **0 puntos**.
 - En cualquier otro caso, obtienes los puntos correspondientes al número del dado.

Por ejemplo:

- Sacar un 6 → 8 puntos.
- Sacar un 4 → 4 puntos.
- Sacar un 1 → 0 puntos.

Después de calcular los puntos de ambos jugadores:

- Si el jugador tiene más puntos → `"¡Has ganado!"`
- Si el ordenador tiene más puntos → `"Ha ganado el ordenador"`
- Si tienen los mismos puntos → `"¡Empate!"`

El programa debe mostrar el resultado de ambos dados, sus puntos y quién ha ganado.

```
Dado del jugador: 6
Puntos del jugador: 8

Dado del ordenador: 4
Puntos del ordenador: 4

¡Has ganado!
```

Reto 2. Mini combate RPG

Vamos a crear un pequeño sistema de combate para un videojuego RPG.

El jugador y el enemigo tendrán:

- **Puntos de vida (PV)**
- **Ataque**
- **Defensa**

El programa debe generar aleatoriamente las características del jugador y del enemigo.

1. Crear los personajes

Para cada personaje, genera:

- PV entre **10 y 20**
- Ataque entre **5 y 10**
- Defensa entre **1 y 6**

Muestra las características de ambos personajes.

2. Resolver el ataque

El jugador realiza un ataque contra el enemigo.

Calcula el daño utilizando:

```
daño = ataque del jugador - defensa del enemigo
```

Pero el daño nunca puede ser negativo.

Por tanto:

- Si el resultado es menor que 0 → el daño será 0.
- Si es mayor que 0 → ese será el daño realizado.

Resta el daño a los puntos de vida del enemigo.

3. Golpe crítico

Genera también un número aleatorio entre 1 y 10.

Si el resultado es **10**, el jugador consigue un golpe crítico y el daño se duplica.

Muestra un mensaje:

```
"¡Golpe crítico!"
```

En caso contrario:

```
"Ataque normal"
```

4. Resultado

Después del ataque:

- Si los PV del enemigo son menores o iguales que 0 → "¡Has derrotado al enemigo!"
- Si todavía tiene PV → "El enemigo sigue con vida"

Muestra también el daño realizado y los PV que le quedan al enemigo.

Restricción: todavía no puedes utilizar bucles ni funciones propias. Debes resolver el combate utilizando las estructuras selectivas y las herramientas de Python que hemos trabajado hasta ahora.

5. Ejemplo de ejecución

```
=== JUGADOR ===
Puntos de vida: 17
Ataque: 9
Defensa: 4

=== ENEMIGO ===
Puntos de vida: 12
Ataque: 6
Defensa: 3

=== ATAQUE ===
Ataque del jugador: 9
Defensa del enemigo: 3
¡Golpe crítico!
Daño realizado: 12
PV restantes del enemigo: 0

¡Has derrotado al enemigo!
```

6. Reto adicional

Añade tres tipos de enemigo:

- **Goblin:** defensa baja, pero mucho ataque.
- **Orcos:** mucho PV y defensa alta.
- **Esqueleto:** características equilibradas.

El programa debe elegir aleatoriamente qué enemigo aparece y aplicar sus características.

Ejemplo de ejecución

```
=== JUGADOR ===
Puntos de vida: 17
Ataque: 9
Defensa: 4

=== ENEMIGO ===
Tipo: Goblin
```

Puntos de vida: 12

Ataque: 9

Defensa: 2

=== ATAQUE ===

Ataque del jugador: 9

Defensa del enemigo: 2

Ataque normal

Daño realizado: 7

PV restantes del enemigo: 5

El enemigo sigue con vida